

Lab Manual — Notes for Teaching Assistants

DSA27 · Weeks 1 to 8 · Data Structures and Algorithms

Dr. Haitham A. El-Ghareeb

Fall 2026

Contents

1. The labs in one page	1
2. Running a session (2 hours)	2
3. What to demonstrate	3
4. Exercise notes	6
5. Checking students' work	17
6. Environment problems you will see	19
7. Answers to the take-home practice	20

*This guide is for the TAs running the labs of weeks 1–8. Students can read it too — there is nothing here they should not see. It contains what to demonstrate, where students go wrong, and how to check their work — not the solutions, which are in **solutions/** for students to use **after** they have tried. Weeks 1–3 teach Python in **Labs/**; from Week 4 the lab is where students build that week's **dsa/** module.*

1. The labs in one page

Week	Lab	Core idea to land	The trap to demonstrate live
1	Interpreter, numbers, strings, lists	A name refers to an object	<code>b = a;</code> <code>b.append(4)</code> changes <code>a</code> the mutable default
2	Control flow, functions, errors, modules	Functions return; errors are information	<code>def f(a, L=[])</code>
3	Data structures, classes, generators	Each structure has a cost; a class bundles data with its contract	the shared class <code>variable tricks = []</code>

Week	Lab	Core idea to land	The trap to demonstrate live
4	Building a dynamic array	Size is not capacity; growing by a factor makes append amortised $O(1)$	a constant growth step: count the copies
5	Building a linked list	Change references in an order that never loses the rest of the list	<code>push_front</code> with its two assignments swapped: a node that points at itself
6	Stacks, brackets and postfix	The top goes where the $O(1)$ end is; the stack is the algorithm	the top-at-index-0 stack that passes every test
7	Queues and the ring buffer	Move the indices, not the data; % makes the block a ring	<code>head += 1</code> with no %, and full against empty
8	Eight ways to search	An invariant for <code>lo</code> and <code>hi</code> , tested on sizes 0, 1 and 2	<code>while lo < hi:</code> which tests fail, and why exactly those

Each lab has a reading (`docs/labs/labNN-*.md`, also as a PDF in `docs/pdf/`), a starter file and a test file: `labs/labNN.py` and `tests/test_labNN.py` for weeks 1–3, then that week's `dsa/` module and its test file — the lab manual's summary table names both, and the `pytest -k` filter when the file is shared.

Where students are coming from. AI students have taken CS1002 Object Oriented Programming. Bio and SWE students have taken CS012, structured programming only, often in C++. Expect the Bio and SWE groups to struggle more in Lab 03 (classes) and the AI group to be overconfident in Lab 01 (they will get the floor-division and aliasing checkpoints wrong anyway). Pair a stronger student with a weaker one for the Checkpoints if the room allows it.

2. Running a session (2 hours)

Time	What	Notes
0:00–0:10	Attendance; environment problems	Attendance counts toward the 75% rule — record it every session.
0:10–0:35	Live demonstration at the REPL	Follow section 3 below. Project a terminal, large font. Type; do not paste.
0:35–1:40	Students work: Checkpoints, then exercises	Walk the room. Ask “what did you predict?” before “what did you get?”

Time	What	Notes
1:40–2:00	Check-off	Each student shows <code>pytest tests/test_labNN.py</code> and explains one line you choose.

Principles for helping.

- Ask them to **read the failing test's output aloud** before you say anything. Half the time they then see it.
- Point to the **docstring** and the **test file** before you give a hint. The habit of reading the specification is the lesson.
- Give the smallest hint that unblocks — the hints in each lab's "Exercises" part are graded from small to large. Do not type on a student's keyboard.
- When a student says "it works on my machine", run the tests. The tests define "works".

3. What to demonstrate

Lab 01 — Week 1

1. Start the REPL; `2 + 2`; `_`; `type(5)`, `type(5.0)`; `8 / 4` is `2.0`.
2. `-7 // 2` and `-7 % 2`. Ask the room first; most will say `-3` and `-1`.
3. `2 ** 100`, then `0.1 + 0.2 == 0.3`. One sentence on why: binary fractions.
4. `word = 'Python'`; index, negative index, slice; draw the "edges between characters" diagram on the board; `word[:-1]`.
5. **The main event.** `a = [1, 2, 3]`; `b = a`; `b.append(4)`; `a`. Then `a is b`, `id(a)`, `c = a[:]`, `c is a`, `c == a`. Draw the arrows on the board.
6. `[[0] * 3] * 3` and set one cell. Leave it unresolved — Lab 03 fixes it.
7. Open `labs/lab01.py`, write `celsius_to_fahrenheit` **wrongly** (no round), run `pytest tests/test_lab01.py -k celsius`, read the failure together, fix it. This is the only exercise you solve in front of them.

Lab 02 — Week 2

1. `if/elif` order: the score `>= 50` before score `>= 90` example.
2. `0` or `'default'`; `[]` and `1/0` — short-circuiting.
3. `for` over a string; `range(5)` is not a list; `enumerate`.
4. The prime-search with `for ... else`. Ask: which statement does the `else` belong to?
5. **The main event.** `def f(a, L=[])` called three times. Then the `None` fix. Then `add_item` versus `replace` — mutation versus rebinding.
6. Run `trace_demo.py` (Part 6.2) and read the traceback bottom-up, aloud.
7. `count_single / count_nested` for `n = 10, 100, 1000` — tie it to Lecture 02.

Lab 03 — Week 3

1. `marks.sort()` returns `None`; `sorted` does not. Everyone has hit this.

2. A list as a stack; then `pop(0)` versus `deque.popleft()`. Optional: time 100,000 `pop(0)` against 100,000 `popleft()`.
3. A comprehension next to the loop it replaces. Then the correct grid: `[[0] * 3 for _ in range(3)]` — closing the loop from Lab 01.
4. `{}` is a dict; `set()` is a set. `x in list` versus `x in set` on a million items with `time.perf_counter`.
5. The counting pattern with `get`, then `setdefault`.
6. **The main event.** Build `Dog` live; then `BadDog` with `tricks = []`. Then `Fraction` with only `__init__` and show `Fraction(1, 2)` prints as `<__main__.Fraction object at 0x...>`; add `__repr__` and show the difference; add `__eq__` and show `==` start working.
7. A three-line generator and `next()` by hand until `StopIteration`.

Lab 04 — Week 4

1. `from dsa.array import Array; a = Array(4); a[4] = 'e'` — the fixed size is the whole problem. Then `a[-1]`: the course `Array` has no negative indices, so the dynamic array must provide them.
2. Draw capacity against size on the board: a block of 8 with 5 used. Run the `show(arr)` helper from Part 0.5 of the manual after each of five appends, and ask the room for the capacity *before* each one appears.
3. Write `_resize` live (it is printed in the lecture): allocate, copy, switch, count. Then write `append` **wrongly** — write before the “is it full?” check — run `pytest tests/test_dynamic_array.py -k "constructor or grows_beyond or doubling"`, read `Array` index 1 out of range for length 1 aloud, and fix it. This is the only method you write in front of them.
4. `DynamicArray(growth=3)` and `growth=10`: print `resize_count` after 1,000 appends. Then `DynamicArray([1, 2], growth=1)` — ask why it fails.
5. The `StepArray` subclass (+ 10). Print `resize_count` for 1,000 appends (100 against 10), then run the Part 6 timing for `n = 500 ... 4,000` and show the two curves bend apart. Do not go bigger live: every element moves through the course `Array`’s bounds check, and the constant-step run grows fourfold per doubling.
6. `pop(0)` against `pop()` on 2,000 elements, timed (about 2 s against a few milliseconds; 8,000 takes tens of seconds per run). One sentence: “this is the queue you build in Week 7”.

Lab 05 — Week 5

1. Build `a = Node(3, Node(7, Node(1)))` at the REPL, from the end backwards. Walk it with `a.next.next.value`; draw the boxes and arrows on the board and label each arrow with the expression that reaches it. Then `draw_linked_list([3, 7, 1], highlight=1)` in the notebook.
2. **The main event.** `push_front` both ways. Write the correct two lines, then swap them in a subclass, push 3 and 7, and show `ll.head.next is ll.head` and `list(itertools.islice(ll, 5))` giving `[7, 7, 7, 7, 7]`. Then `list(ll)` and `Ctrl+C`. The rule: link the new node in before you let go of the old one.
3. `remove` on `3 → 7 → 1 → 9` at the whiteboard: the loop looks at `prev.next`, so it stays on the node before. Then ask the room how to remove the 3 — there is no `prev`: the head special case.

4. Trace reverse on $1 \rightarrow 2 \rightarrow 3 \rightarrow 4$ with three coloured markers (prev, node, the saved next). After each step, draw the **two** chains separately. Ask what happens if the saved-next line is dropped.
5. Time for `i in range(len(l1)): l1[i]` against for `v in l1` for $n = 1000$ and 2000 with `time.perf_counter` (lists built with `push_front`). Four times against twice. "An interface does not tell you the cost."
6. Close with a stack: `push_front` three brackets, `pop_front` them back in reverse — the bridge to Lecture 06.

Lab 06 — Week 6

1. `pytest tests/test_stack_queue.py -k stack --collect-only -q` — it collects **all 18** tests, queues included, because `-k` also matches the file name `test_stack_queue`. Then show the filter the lab and Lecture 06 use, `-k "lifo or peek or empty_behaviour or balanced or infix"` (12 tests). Two minutes that save the room an hour of chasing queue failures.
2. Write `push` and `pop` live on the `DynamicArray`, top at the **end**. Then write `FrontStack` (`insert_at(0, ...)` and `pop(0)`), run the three stack tests on it — they pass — and ask the room what is wrong with it. Run Part 2's measurement with sizes up to 1,600 only: the index-0 line quadruples per doubling.
3. Whiteboard: the three unbalanced inputs `([]), )` and `(, and` which check each one needs. Leave the code to them.
4. Whiteboard: shunting-yard on `8 - 3 - 2` with two columns, output and operator stack. Stop at the second `-` and ask: does the first `-` leave? Then evaluate the postfix both ways (3 against 7) to show why it must — and point out the lecture's rule 2 ends "stopping at a `(`".
5. `8 3 -` at the board: `pop`, `pop`, and which one is the left operand. Name them **right** and **left** on the board, in the order they are popped.
6. Show `'3+4'.split()` at the REPL: one token. One sentence on why Week 15 starts with a tokenizer.

Lab 07 — Week 7

1. `SlowQueue` in two lines at the REPL, then time one `pop(0)` on a `DynamicArray` of 1,000 and of 4,000 items. Ask the room why it is four times slower, and why swapping the ends does not help.
2. **The main event.** On the whiteboard, draw a 4-slot block and run Lecture 07's capacity-4 trace (A to F) with the room calling out `head`, `size` and the tail (`head + size`) `% 4` after each step. Stop at "enqueue E" and ask where it goes before you write it.
3. Show the start state and the full state side by side: `head == tail` in both. Ask how the class tells them apart; point at `is_empty` and `is_full`.
4. Read the given `CircularQueue.__iter__` aloud — `(head + offset) % capacity` — and say that `enqueue`, `dequeue` and the growth copy all reuse that formula.
5. Show the `-k` pitfall: `pytest tests/test_stack_queue.py -k queue --collect-only -q` collects all 18 tests, because `-k` also matches the file name `test_stack_queue.py` — so no filter may contain "queue" or "stack". Then the filter the lab and Lecture 07 use: `-k`

"fifo or dequeue or circular" (6 tests).

6. The growth trap on the board: the full ring E F C D, head 2, copied slot by slot into 8 slots — read it from head 2 and let the room find E and F "lost". Then the unrolled copy. Do not write the code.

Lab 08 — Week 8

Midterm week: keep the demonstration to 20 minutes and the room on Parts 1–5.

1. On the board, trace `binary_search([4, 9, 13, 20, 26, 31, 37, 44], 10)` as a lo/hi/mid table. Stop on the last row (lo = 2, hi = 1) and ask what the 2 means: the insertion point, which is `lower_bound`.
2. Ask the room for `binary_search([3, 8], 8)` by hand, loop turns included. Then say the rule: test sizes 0, 1 and 2 before running `pytest`.
3. **The main event.** With a working `binary_search` projected, change `lo <= hi` to `lo < hi` and ask which of the 11 tests from `pytest tests/test_searching.py -v -k "binary_search and not recursive"` will fail. Collect guesses, then run it: exactly 4 fail and 7 pass. The 4 are target 9 in `[1, 3, 5, 7, 9]`, the empty/single test, first/last, and the million-element test. The three outside-the-values tests pass because they are misses, and the random-data test passes because only 2 of its 30 targets are present. Put the `=` back.
4. Change `lo = mid + 1` to `lo = mid` and run again: the run hangs on one test. Show `Ctrl+C` and read the traceback — it points at the `while` line.
5. Write `[1, 2, 2, 2, 5]` on the board with L/R marks for `< 2` and for `<= 2`: `lower_bound` and `upper_bound` are the same search for the first R. Say "half-open" and why `lo < hi` is right **there**.
6. Warn about the filter trap: `-k linear` selects 8 tests; `-k "linear_search and not agrees"` selects the 2 for `linear_search`.

4. Exercise notes

For each exercise: what it tests, and the mistakes you will see most. The tests are the definition of correct — when in doubt, read `tests/test_labNN.py`.

Lab 01

Exercise	What it is really testing	Common mistakes
<code>seconds_to_hms</code>	<code>//</code> and <code>%</code> twice; <code>:02d</code>	<code>seconds / 3600</code> (a float); padding the hours; forgetting that minutes are <code>seconds % 3600 // 60</code> , not <code>seconds // 60</code>
<code>celsius_to_fahrenheit</code>	float arithmetic, <code>round(x, 1)</code>	printing instead of returning; <code>9 // 5</code> (integer division gives 1)

Exercise	What it is really testing	Common mistakes
<code>split_evenly</code>	returning two values	returning a list <code>[a, b]</code> — the test compares with a tuple, and <code>[33, 1] != (33, 1)</code>
<code>initials</code>	<code>split()</code> with no argument; building a string in a loop	<code>split(' ')</code> , which produces empty strings for repeated spaces and then <code>' '[0]</code> raises <code>IndexError</code>
<code>is_palindrome_word</code>	<code>strip</code> , <code>lower</code> , <code>[::-1]</code>	returning <code>'True'</code> (a string) or <code>1</code> ; the test uses <code>is True</code>
<code>mask_email</code>	<code>index('@')</code> , slicing, <code>'*' * n</code>	off-by-one in the number of stars; forgetting the short-name case
<code>middle</code>	slice arithmetic	a chain of <code>ifs</code> that breaks for length 0 or 1; the one-slice answer exists
<code>rotate_left</code>	<code>%</code> , two slices, not mutating	<code>k % 0</code> on the empty list; mutating the input with <code>pop/append</code> — a test checks the original is unchanged
<code>fib_list</code>	the tutorial loop, collecting into a list	an off-by-one: <code>n = 1</code> must give <code>[0]</code>
<code>collatz_steps</code>	a <code>while</code> with an <code>if</code> inside	counting the starting number as a step; <code>n / 2</code> (a float — it still terminates, but the habit is wrong)

Rule for Lab 01: no `for`, no `import`. A student who uses `for` from prior knowledge has not failed — but ask them to rewrite one function with `while`, because the point of the lab is the `while`-loop mechanics they will need for binary search in Week 8.

Lab 02

Exercise	What it is really testing	Common mistakes
<code>classify_triangle</code>	validation before classification; raise <code>ValueError</code>	<code>>=</code> versus <code>></code> in the inequality (1, 2, 3 is degenerate and must be rejected); checking only one of the three inequalities; returning an error string

Exercise	What it is really testing	Common mistakes
course_result	order of conditions	checking the total before attendance; <code>final < 30</code> (it is 30% of 60 , i.e. 18); not validating ranges
fizzbuzz	for, %	testing % 3 before % 15, so 15 becomes "Fizz"; returning ints instead of strings
is_prime	early return; the $\sqrt{n}$ bound	<code>range(2, int(n ** 0.5))</code> — misses the square root itself, so 9 and 25 come out prime; forgetting <code>n < 2</code>
primes_below	reuse	re-implementing the primality test inline
first_repeated	nested loop, early exit	returning the first value that <i>has</i> a duplicate somewhere later, instead of the value whose <i>repeat comes first</i> : for <code>[2, 5, 5, 2]</code> that gives 2, but the answer is 5. Ask them to trace it by hand
count_pairs	counting, then a closed form	the formula <code>n * (n - 1) / 2</code> returns a float ; they need <code>//</code> . Both loop and formula pass — ask which is $O(1)$
calculator	match, capture patterns, <code>*rest</code>	case <code>["sum", *rest]</code> accepts "sum" with no numbers — the test requires <code>ValueError</code> ; a bare <code>except:</code> that also swallows their own bugs
parse_int	<code>try/except ValueError</code>	<code>text.isdigit()</code> checks (fails on <code>" -7 "</code>); catching <code>Exception</code>
stats	<code>*args</code>	starting min/max at 0; using min/max (not allowed); integer division for the mean
append_to	the mutable default	writing <code>target=[]</code> — the second test call fails, which is exactly the lesson
make_multiplier	returning a function	returning <code>k * x</code> with <code>x</code> undefined; calling the lambda instead of returning it

Exercise	What it is really testing	Common mistakes
<code>apply_n</code>	passing a function	calling <code>f</code> once; recursion without a base case for <code>n = 0</code>

Lab 03

Exercise	What it is really testing	Common mistakes
<code>unique_in_order</code>	set for membership, list for order	if <code>v</code> not in <code>result</code> — $O(n^2)$; the 400,000-item test takes minutes. Let them see it hang, then ask why. <code>list(set(values))</code> loses the order
<code>word_frequencies</code>	<code>split</code> , <code>strip(chars)</code> , <code>get</code>	<code>replace</code> to delete punctuation (also removes it from inside words); not skipping words that become empty
<code>top_k</code>	sorting by a tuple key	sorting by count only (ties come out in dictionary order); <code>reverse=True</code> on a tuple key reverses the alphabetical tie-break too
<code>transpose</code>	nested comprehension	the order of the two fors; the empty matrix (<code>matrix[0]</code> raises)
<code>invert</code>	grouping	overwriting instead of appending (<code>out[v] = k</code>); forgetting to sort the key lists
<code>common_elements</code>	<code>&</code>	returning a set instead of a sorted list
<code>group_by_length</code>	<code>setdefault</code> or <code>get</code>	sorting the words (the test expects input order)
<code>Bag</code>	a class with state; dunders	see below
<code>countdown</code>	<code>yield</code>	building and returning a list — the test checks it is a generator
<code>chunks</code>	a generator with <code>range(0, n, size)</code>	no explicit size check: <code>range(0, n, 0)</code> happens to raise <code>ValueError</code> for size 0, but a negative size silently yields nothing — the test checks both; returning a list

The Bag class is the most important exercise of the three labs, because it is the shape of every dsa/ class. Check, in this order:

1. The counts live in a **dictionary created in `__init__`**, not a class variable. (The test that two bags do not share state catches the class-variable version.)
2. `__len__` is $O(1)$: a running total, not `sum(self._counts.values())`. The tests cannot detect the $O(k)$ version — so ask the student what `len` costs. This is a “know-why” question, the kind the exam asks.
3. `remove` deletes the key when a count reaches zero, and raises `KeyError` — not `ValueError`, not `print`.
4. `__eq__` compares the dictionaries, and returns `NotImplemented` for non-bags.
5. `__repr__` is `Bag([...])` with items **sorted**, repeats included.

Lab 04

Method	What it is really testing	Common mistakes
<code>_resize</code>	allocate, copy size elements, switch; keeping <code>_capacity</code> and <code>resize_count</code> in step	forgetting to assign <code>self._block = block</code> (the writes go to the old block); not updating <code>_capacity</code> ; no <code>resize_count += 1</code> (<code>resize_count</code> never went up – increment it in <code>_resize</code>); copying into a Python list first (storage rule)
<code>append</code>	resize before writing; multiplying by <code>self.growth</code>	writing first, then checking (Array index 1 out of range for length 1); <code>self._capacity + self.growth</code> (only the <code>resize-count</code> test fails: 500 reallocations); hard-coding <code>* 2</code> and ignoring <code>growth</code>
<code>__getitem__</code>	bounds against size, negative translation by size	comparing with <code>_capacity</code> (<code>[1, 2, 3][3]</code> returns <code>None</code> : DID NOT RAISE); <code>index += self._capacity</code> ; letting the Array raise for negatives
<code>__setitem__</code>	same shape as <code>__getitem__</code>	no bounds check at all — writes an unused slot silently (<code>test_setitem_out_of_range_raises</code> : DID NOT RAISE)

Method	What it is really testing	Common mistakes
<code>insert_at</code>	<code>IndexError</code> unless <code>0 <= index <= size</code> , checked before anything moves; resize when full; shift right from the end	shifting from the front (<code>[1, 99, 2, 2]</code>); forgetting <code>_size += 1</code> ; no resize when full (<code>test_insert_at_grows_when_full</code> : Array index 1 out of range for length 1); clamping like <code>list.insert</code> or translating negatives (DID NOT RAISE); no index check at all (<code>test_insert_at_out_of_range_raises</code> and <code>[-1]</code> : DID NOT RAISE; <code>[-1]</code> still passes, because the Array raises mid-shift, after corrupting the array to <code>[1, 1, 2, 3, 4]</code>)
<code>pop</code>	empty check, negative translation, shift left from the front, clear the freed slot	no translation of the default <code>-1</code> ; shifting the wrong way (<code>assert [1] == [2]</code>); not clearing the slot (<code>test_pop_clears_the_freed_slot</code> : <code>assert 'b' is None</code>); loop <code>range(index, self._size)</code> — crashes only on a full array, and no test catches it

The method that matters most is `append`, and in particular the choice of `capacity * growth`. It is three lines, and the only one of them that matters for the exam is the multiplication: ask every student what would change with `+ 2`, and expect “it still works, but resizes every other `append`, so n appends cost $\Theta(n^2)$ ”. Students who pass all 19 tests often cannot say this. Also ask to see *where* `insert_at` checks its index: it must be the first thing it does. A check placed after the shifting loop fails `test_insert_at_out_of_range_raises[-1]` (“check the index BEFORE shifting anything”): the course Array raises for index `-1` mid-shift, after the values have already moved.

Lab 05

Method	What it is really testing	Common mistakes
push_front	the order of two reference assignments; <code>_size</code>	setting head first — a self-loop, and the test hangs with no output; forgetting <code>_size += 1 (assert 0 == 3)</code>
append	the empty list; stopping on the last node	no empty case — <code>AttributeError: 'NoneType' object has no attribute 'next'</code> in 19 of the 24 tests, because <code>__init__</code> calls <code>append</code> ; while node is not None walks off the end (the message ends "and no dict for setting new attributes"); <code>_size</code> in one branch only
insert_at	stopping at index - 1; reusing <code>push_front</code> ; the <code><=</code> bound	walking index steps (the new node lands one place right); no validation (-1 quietly inserts after the head: DID NOT RAISE); <code>_size</code> added twice when index 0 calls <code>push_front</code>
pop_front	the empty check first	reading <code>self.head.value</code> on an empty list (<code>AttributeError</code> , not <code>IndexError</code>)
remove	the head special case; looking one node ahead	no head branch (<code>assert False is True</code> on <code>remove(1)</code>); forgetting <code>_size -= 1</code> in one or both branches — only <code>test_remove_updates_the_length</code> catches it (<code>assert 2 == 1</code> or <code>assert 3 == 1</code>)
find	the traversal loop that visits every node	while <code>node.next</code> is not None never checks the last node (<code>assert -1 == 2</code>) and crashes on an empty list

Method	What it is really testing	Common mistakes
<code>__getitem__</code>	range check before walking	no check: <code>AttributeError</code> instead of <code>IndexError</code> ; then calling <code>self[i]</code> in a loop inside their own methods — $O(n^2)$ that no test detects
<code>reverse</code>	holding an invariant; saving <code>next</code> first	stepping with <code>node = node.next</code> after re-linking (list becomes <code>[1]</code>); forgetting <code>self.head = prev</code> (also <code>[1]</code> , with <code>len</code> still 4); rebuilding with new nodes, or writing the values back into the old nodes (either way only the relinks test fails — the second with “the old last node must become the head: relink, do not copy values”)

reverse is the method that matters most. It is the one that cannot be written by pattern-matching the lecture’s code: the student must hold the invariant — `prev` heads the reversed part, `node` heads the rest — and see why the next node must be saved before the link is turned. Insist on the paper trace (the table in Part 7) before any code; a student who has filled it in writes the loop in a minute, and one who has not will try orders at random until the tests pass. Watch for shortcuts through a Python `list`: building new nodes, or writing the reversed values back into the existing nodes. The relinks test now catches both — the second by checking that the old last node object became the head — but a student who hit that message should be able to say why copying values is not reversing a linked list. Ask them to point at the line that turns a link round.

Lab 06

Method	What it is really testing	Common mistakes
<code>Stack.push</code>	the top at the end of the <code>DynamicArray</code>	<code>insert_at(0, value)</code> — passes every test, $O(n)$; a Python list as <code>self._items</code> (storage rule)
<code>Stack.pop</code>	same end as push; the empty contract	<code>self._items.pop(0)</code> (a queue: <code>assert 1 == 3</code>); returning <code>None</code> when empty (DID NOT RAISE <code>IndexError</code>)

Method	What it is really testing	Common mistakes
<code>Stack.peak</code>	reading without changing	calling <code>pop</code> (length drops: <code>assert 1 == 2</code>); <code>self._items[0]</code> (the bottom); <code>self._items[len(self._items)]</code> (<code>IndexError: 2</code>)
<code>is_balanced</code>	all three checks	no partner check (<code>(([]))</code> gives <code>True</code>); no non-empty check (<code>()</code> raises <code>IndexError: pop from empty stack</code>); return <code>True</code> at the end (<code>()</code> gives <code>True</code>); a Python list as the stack
<code>evaluate_postfix</code>	operand order; malformed input	swapped operands (<code>assert -2 == 2</code> ; the exact-division and division-by-zero tests fail too); no count check before popping (<code>IndexError</code> instead of <code>ValueError</code>); no "exactly one left" check; <code>//</code> instead of <code>/</code> (<code>assert 3 == 3.5</code>)
<code>infix_to_postfix</code>	equal precedence pops; (as a barrier	<code>></code> instead of <code>>=</code> — gives <code>8 3 2 - -</code> , caught only by <code>test_infix_to_postfix_is_left_associative</code> ; asking the precedence table about (<code>(</code> (<code>KeyError: '('</code>); (not discarded (it appears in the output); no flush at the end

infix_to_postfix matters most. Its first test never puts two operators of equal precedence side by side, so only `test_infix_to_postfix_is_left_associative` (`8 - 3 - 2, 8 / 4 / 2`) catches the `>-for->=` bug, with `assert ['8', '3', '2', '-', '-'] == ['8', '3', '-', '2', '-']`. When a student sees that message, ask them to evaluate both lists by hand (3 against 7) before they touch the code. Neither test has two pairs of parentheses, so also ask for `'( 1 + 2 ) * ( 3 - 4 ) / 5'.split()` in front of you: `1 2 + 3 4 - * 5 /`, which their `evaluate_postfix` makes `-0.6`. Also watch for `test_postfix_pairs_with_the_stack_exercise` in `tests/test_translation.py`: it needs the Week 15 `tokenize`, so students who run `-k postfix` see one failure that is not theirs. The lab and the lecture use `-k evaluate_postfix` (11 tests).

Lab 07

Method	What it is really testing	Common mistakes
<code>SlowQueue.enqueue</code>	using the <code>DynamicArray</code> through its public methods	reaching into <code>self._items._block</code> ; adding a Python list beside it
<code>SlowQueue.dequeue</code>	which end is the front; raising on empty	<code>pop()</code> (a stack: <code>assert 3 == 1</code>); return <code>None</code> when empty (DID NOT RAISE <code>IndexError</code>); forgetting <code>return</code>
<code>CircularQueue.enqueue</code>	the tail formula with <code>%</code> ; full by size	no <code>%</code> (Array index 3 out of range for length 3 in the wraps test); full tested as <code>tail == head</code> , so the first enqueue raises “queue is full”; forgetting <code>size += 1</code>
<code>CircularQueue.dequeue</code>	wrapping head; order of read and clear	<code>self._head += 1</code> without <code>%</code> — passes the wraps test, fails only the interleaved one with Array index 4 out of range for length 4; clearing the slot before reading it (<code>assert None == 1</code>); no empty check
growth (challenge)	copying in queue order; resetting three fields	the naive slot-i-to-slot-i copy; forgetting <code>head = 0</code> or the new capacity; computing the slot before growing; <code>2 * 0</code> for <code>CircularQueue(0)</code>

`CircularQueue.dequeue` matters most, because its classic bug hides: a head that never wraps still passes three of the four `CircularQueue` tests, including the one named “wraps around” — that test dequeues once and reads the queue through the given `__iter__`, which does its own `%`. Only `test_circular_queue_interleaved_operations` reaches the fifth dequeue, at `block[4]`. When a student says “the wrap test passes, so my wrap is right”, ask them to trace the interleaved test by hand. For the growth challenge the provided tests prove nothing (they never overfill a ring): check that the student wrote their own test on a **wrapped** ring (Part 6.4 of the lab), not only on a ring with head 0, where the naive copy happens to work.

Lab 08

Method	What it is really testing	Common mistakes
<code>linear_search</code>	the loop shape; return <code>-1</code> after the loop; the first match	return <code>-1</code> inside the loop (assert <code>-1 == 2</code> , and assert <code>-1 == 1</code> in the first-match test); returning the last match (assert <code>4 == 1</code> in <code>test_linear_search_returns_the_first</code>)
<code>binary_search</code>	an inclusive range kept consistent	<code>lo < hi</code> (4 tests fail); <code>lo = mid / hi = mid</code> (a test hangs); <code>/ for //</code> (<code>TypeError: list indices must be integers or slices, not float</code>); no final return <code>-1</code> (assert <code>None == -1</code> , 6 tests); <code>hi = len(values)</code> (<code>IndexError</code> , 4 tests, including the outside targets 10 and 99)
<code>binary_search_recursive</code>	recursion with <code>lo/hi</code> parameters; a default that depends on another argument	slicing (assert <code>1 == 4</code> : the index is into the slice); a call without return (assert <code>None == ...</code> , 9 of 10 fail); if not <code>hi</code> : instead of if <code>hi is None</code> : — only the random test fails, with <code>RecursionError</code>
<code>lower_bound / upper_bound</code>	a half-open range, and never stopping early	inclusive <code>hi = len - 1</code> (assert <code>4 == 5: len(values) unreachable</code>); returning <code>-1</code> past the end; <code><</code> in <code>upper_bound</code> (it becomes <code>lower_bound</code> : assert <code>1 == 4</code> , and assert <code>0 == 4</code> in the duplicates test); <code>lo <= hi</code> with <code>hi = mid</code> hangs
<code>jump_search</code>	<code>math.isqrt</code> ; the short last block	<code>n ** 0.5</code> (a float index: <code>TypeError</code>); walking to <code>prev + step</code> without <code>min(..., n)</code> — <code>IndexError</code> only in <code>test_target_outside_the_values</code> for 10 and 99, the targets that walk off the short last block

Method	What it is really testing	Common mistakes
exponential_search	slot 0, doubling from 1, capping the bound	bound = 0 (hangs: 0 * 2 is 0); no bound < n before values[bound] (IndexError, 5 tests); hi = bound uncapped (IndexError, 4 tests)
interpolation_search	the three traps of the lecture	no equal-ends check (ZeroDivisionError: integer division or modulo by zero on [42]); no out-of-range check — hangs on 4 in [1, 3, 5, 7, 9], because the guess lands left of lo; / in the formula (TypeError, 6 of 10 fail)

binary_search is the one that matters most — for the lab, for the final, and for every language the students will use after this course. The tests catch the loud bugs, but check two things by reading the code. First, that the student can state what lo and hi mean (both inclusive) and derive $lo \leq hi$ and $mid \pm 1$ from that, rather than remembering them; ask them to explain why the bounds' loop uses $lo < hi$ and $hi = mid$ and theirs does not. The $lo < hi$ bug still passes 7 of the 11 tests, so reading matters. Second, that values[mid] is read once per turn and that nothing slices or calls in, index or bisect — the million-element test catches a linear scan, but not a stray slice in the recursive version (the slicing version fails only because it loses the offset). A student who passes all 11 and cannot trace [3, 8] for 8 by hand has not finished.

5. Checking students' work

From the student's DSA27 folder, with their virtual environment active:

```
git status                # nothing unexpected changed?
git diff --stat tests/    # tests must be unchanged – this must be empty
pytest tests/test_lab01.py -q  # the result you record
```

If tests were edited, restore them with `git checkout -- tests/` and run again. Record the result of the original tests.

Suggested marking, per lab (the lab portion of coursework is announced by the lecturer):

Mark	Meaning
2	All tests pass, and the student explains the line you point at
1	Most tests pass, or all pass but the explanation is weak
0	Little working, or the student cannot explain their own code

The explanation matters more than the count. A student who passes 8 of 10 and can explain every line has learnt more than one who passes 10 of 10 with code they cannot read.

Know-why questions, labs 04–08

From Week 4 the tests check behaviour; these questions check understanding. Ask one about the student's own code at check-off.

Lab 04 — know-why questions

- *"Why does `append` resize before it writes, not after?"* — Slot size of a full block does not exist; writing first raises `IndexError`.
- *"What would `resize_count` be after 1,000 appends if you wrote `+` instead of `*`? Why does it matter?"* — 500 with growth 2 (100 with a step of 10); the copies become $\Theta(n^2)$ in total, so `append` is amortised $O(n)$, not $O(1)$.
- *"Why do you set the freed slot to `None` in `pop`?"* — Otherwise the block still refers to the removed object and the garbage collector cannot free it.
- *"Why does `insert_at(4, x)` on a three-element array raise, when `insert_at(3, x)` does not?"* — Valid positions are $0 \dots \text{size}$; inserting at `size` appends, anything past it would leave a gap.
- *"What does `pop(0)` cost, and which structure next month pays it?"* — $O(n)$: every other element shifts left; `SlowQueue` in Week 7 dequeues with it.

Lab 05 — know-why questions

- *"Why must `new.next = self.head` come before `self.head = new` — and why does `self.head = Node(value, self.head)` not have the problem?"* — The other order makes the node point at itself and loses the list; in the one-liner the right-hand side, including the old head, is evaluated before head changes.
- *"Your `remove` has a separate branch for the head. Why can't the loop handle it?"* — The loop changes `prev.next`, and the head has no node before it; only `self.head` refers to it (a sentinel would remove the case).
- *"Point at the line in `reverse` that would lose the list if it were missing."* — The line that saves `node.next` before `node.next = prev` overwrites it.
- *"What does `len(LL)` cost, and what would break if you deleted `_size -= 1` from `remove`? Would the tests notice?"* — $O(1)$ because `_size` is kept up to date; `len` would over-count after every removal, and `test_remove_updates_the_length` would fail — the only `remove` test that checks `len`.
- *"What does `for i in range(len(LL)): print(LL[i])` cost, and what do you write instead?"* — $O(n^2)$, since every `ll[i]` walks from the head; `for v in ll:` is $O(n)$.

Lab 06 — know-why questions

- *"What does your `push` cost, and why?"* — $O(1)$ amortised: it appends at the end of the `DynamicArray`; nothing shifts, and the occasional `resize` averages out.
- *"Your tests pass with the top at index 0 too. Why is that version wrong?"* — every `push` and `pop` shifts all n items: $O(n)$ each, $O(n^2)$ for n operations; the tests check behaviour, not cost.

- “Show me the line in `is_balanced` that makes `)` return `False`.” — the non-empty check before popping; without it the pop raises `IndexError`.
- “In `evaluate_postfix`, which popped value is the left operand?” — the second one popped; for `8 3 -`, 3 comes off first and the answer is `8 - 3`.
- “Why `>=` and not `>` in shunting-yard?” — equal precedence must leave first so that `-` and `/` are left associative: `8 - 3 - 2` is `(8 - 3) - 2`.

Lab 07 — know-why questions

- “Why does your `SlowQueue.dequeue` cost $O(n)$, and would putting the front at the end fix it?” — `pop(0)` shifts every remaining item; swapping the ends makes `enqueue` `insert_at(0, x)`, which shifts instead.
- “Where does your next `enqueue` write, and why the `%`?” — at `(head + size) % capacity`; without `%` the index runs past the end of the `Array` instead of reusing the freed slots at the front.
- “Why do you store `_size`? Could you test full with `tail == head`?” — no: `tail == head` is true both when empty and when full; the size tells them apart.
- “Why set the dequeued slot to `None` — no test checks it?” — otherwise the block keeps a reference to the dequeued object and Python cannot free it.
- “When your ring grows, why can’t you copy slot `i` to slot `i`?” — once the data has wrapped, the front is not at slot 0; copy the `i`-th queue item from `(head + i) % old_capacity` to slot `i`, then set `head` to 0 and the new capacity.

Lab 08 — know-why questions

- “Your loop is `while lo <= hi`. What goes wrong with `<`?” — The one-element range is never examined, so the last position (and others) can never be found.
- “Why is `lo < hi` correct in your `lower_bound`?” — There `hi` is excluded (half-open), so `lo == hi` already means an empty range.
- “Why does your recursive version take `lo` and `hi` instead of slicing?” — A slice copies ($O(n)$ in total) and its indices are relative to the slice.
- “What is `upper_bound(v, x) - lower_bound(v, x)`, and what does it cost?” — The number of copies of `x`, in $O(\log n)$ whatever that number is.
- “When is your interpolation search slower than linear search?” — On skewed values: every guess lands near `lo`, one element per step, three reads each.

6. Environment problems you will see

Symptom	Cause	Fix
python opens the Microsoft Store	Windows App Execution Aliases	<i>Settings</i> → <i>Apps</i> → <i>Advanced app settings</i> → <i>App execution aliases</i> : switch off <code>python.exe</code> and <code>python3.exe</code>

Symptom	Cause	Fix
pytest is not recognised	venv not activated	<code>.\.venv\Scripts\activate</code> ; the prompt must start with (.venv)
activate refused: "running scripts is disabled"	PowerShell execution policy	<code>Set-ExecutionPolicy -Scope CurrentUser RemoteSigned</code> , then activate again
ModuleNotFoundError: No module named 'labs'	running from the wrong folder	cd to the DSA27 folder — the one with <code>pytest.ini</code> in it
VS Code runs a different Python	wrong interpreter selected	<code>Ctrl+Shift+P</code> → <i>Python: Select Interpreter</i> , then <code>.\.venv\Scripts\python.exe</code>
Tests pass in the terminal but not in VS Code (or the reverse)	two interpreters	as above; <code>python -c "import sys; print(sys.prefix)"</code> in both
Changes to <code>labs/lab01.py</code> have no effect in the REPL A test "hangs"	the module was already imported an infinite while loop, or $O(n^2)$ in <code>unique_in_order</code>	<code>exit()</code> and restart the REPL, or just use <code>pytest</code> <code>Ctrl+C</code> ; the traceback shows the line it was on
graphviz tests fail in pytest <code>-m "not challenge"</code>	the dot binary is not installed or not on PATH	install Graphviz from https://graphviz.org/download/ and reopen the terminal. Not needed for the labs themselves
<code>git clone</code> asks for a password	HTTPS cloning of a public repo should not — they probably mistyped the URL	copy the URL from the README

7. Answers to the take-home practice

The take-home items are ungraded and discussed at the start of the following lab. The quick answers:

- **Lab 01, item 3:** `x` is `[1, [2, 3, 4]]` — `y[0] = 9` changed only the copy; the inner list is shared.
- **Lab 01, item 5:** the time roughly doubles when the input doubles — linear.
- **Lab 02, item 3:** 20 iterations for 1,000,000 — one per binary digit, $\text{floor}(\log_2 n) + 1$.
- **Lab 02, item 4:** 7 guesses — $\text{ceil}(\log_2 101)$ — the same halving as Lab 01's Checkpoint 4 and, in Week 8, binary search.
- **Lab 02, item 5:** yes, a straight line for primes: $O(n)$.
- **Lab 03, item 1:** sorting is $O(n \log n)$; counting (dictionary or Bag) is $O(n)$.
- **Lab 03, item 3:** `zip` stops at the shorter iterable, and a generator only computes a value

when asked, so only ten are ever produced.

- **Lab 03, item 5:** slicing copies the rest of the list at every call, so the sliced version is $O(n^2)$ in total and the index version $O(n)$.